

# UFCS-FQL Hybrid Protocol vs. Industry Video Delivery

Oct 5, 2026 · @Garry Howard

## The short answer

Our hybrid protocol does not make video files smaller than the industry does. It uses the same video "shrinking" tools everyone uses (H.264, H.265, AV1), so the picture looks the same and the file is the same size.

Where it is different is in **how the video travels**. Think of the video as the cargo and our protocol as the delivery truck:

- **Same cargo, same size.** Storage and picture quality match the industry exactly.
- **The packaging is about as light as the best industry option.** In total it adds 0.4% to the video, against 0.3% for a standard MP4 and 4.7% for old MPEG-TS.
- **A single big download is a little slower than plain web delivery** in our test (0.70 s vs 0.60 s for a 3.5 MB clip).
- **Live video is where we win.** Separate lanes for urgent and bulk traffic kept live video arriving in 0.14 s instead of 0.60 s when the line was busy. Industry web streaming usually runs 2 to 30 seconds behind live.
- **Every delivery is checked.** Every piece is fingerprinted, and the copy that arrives is proven identical to the original.

In one line: industry tools win on making the cargo small; our protocol wins on getting urgent cargo there first and proving it arrived untouched.

## What is being compared, layer by layer

Video delivery has four layers. Our protocol replaces only the middle two, so a fair comparison goes one layer at a time.

| Layer                    | What it does, in plain words                       | Industry standard               | QueryBook UFCS-FQL hybrid                                                  |
|--------------------------|----------------------------------------------------|---------------------------------|----------------------------------------------------------------------------|
| 1. Shrinking (codec)     | Squeezes the picture so it takes less space        | H.264, H.265, AV1               | The same H.264, H.265, AV1                                                 |
| 2. Packaging (container) | Puts video and sound in a box with labels          | MP4, MPEG-TS, FLV               | The same boxes, or our own 32-byte label per piece                         |
| 3. Delivery method       | Decides what is sent, in what order, on which lane | HLS, DASH (web downloads), RTMP | Three priority lanes (urgent, normal, bulk) with speed limits and receipts |
| 4. The road (network)    | Moves the bytes                                    | TCP/IP                          | The same TCP/IP                                                            |

Because layers 1 and 4 are shared, any difference in file size or picture quality would be a mistake. The real differences live in layers 2 and 3.

## Compression: how small the video gets

It's a tie by design: both sides use the same video shrinkers. Raw video shrank 102 to 131 times in our tests, whichever way it was delivered.

What our protocol adds is **choosing the right shrinker for each kind of content automatically**:

| Content                             | What our protocol picks                      | Result in our tests                                                     |
|-------------------------------------|----------------------------------------------|-------------------------------------------------------------------------|
| Video                               | H.264, H.265 or AV1, sized to the line speed | Same as industry at the same settings                                   |
| Sound                               | Opus (speech drops to a lighter setting)     | Same as industry                                                        |
| Pictures                            | WebP                                         | Same as industry                                                        |
| Data and captions (facts, metadata) | Zstd or our fact-dictionary shrinker         | About 6.7 times smaller, vs about 5 times with the usual web zip (gzip) |

The one real compression gain is on the data that travels with the video, not on the video itself. For most streaming services that data is a small share of the bill.

Picking a newer shrinker matters far more than the delivery method. At the same bitrate on fast-moving footage, AV1 scored 0.917 on the standard picture-match scale (SSIM, where 1.0 is a perfect copy) against 0.855 for H.264. Our protocol can carry any of them; so can the industry.

## Throughput: how fast it moves

For one big file on a quiet line, plain web delivery was about 15% faster. When urgent and bulk traffic share a busy line, our protocol got live video through about 4 times sooner.

**Test 1: one file, quiet line.** The same 3.5 MB video file (10 seconds of 720p) went over the same simulated 50 Mbps line with a 20 ms delay each way. Each was run 3 times.

| Method                                           | Time to deliver | Effective speed | Share of the 50 Mbps line |
|--------------------------------------------------|-----------------|-----------------|---------------------------|
| Plain web download (HTTP, what HLS and DASH use) | 0.60 s          | 46.5 Mbps       | 93%                       |
| QueryBook UFCS-FQL                               | 0.70 s          | 39.8 Mbps       | 80%                       |

Why web delivery won: it opens one connection and simply pours the file down it. Ours opens three lanes and waits for a signed-for receipt before calling the job done. Our test nodes are also written in PHP, which is slower than the web server's C code. On longer

transfers in our earlier tests, our protocol reached 89 to 95% of the line, close to the web's figure.

**Test 2: live video plus a big download on a busy long-distance line** (20 Mbps, 80 ms delay each way).

| Setup                                                         | Live video arrival time<br>(typical) | Worst<br>5% |
|---------------------------------------------------------------|--------------------------------------|-------------|
| Everything in one connection (how one web connection behaves) | 0.60 s                               | 0.64 s      |
| Our three priority lanes                                      | 0.14 s                               | 0.18 s      |

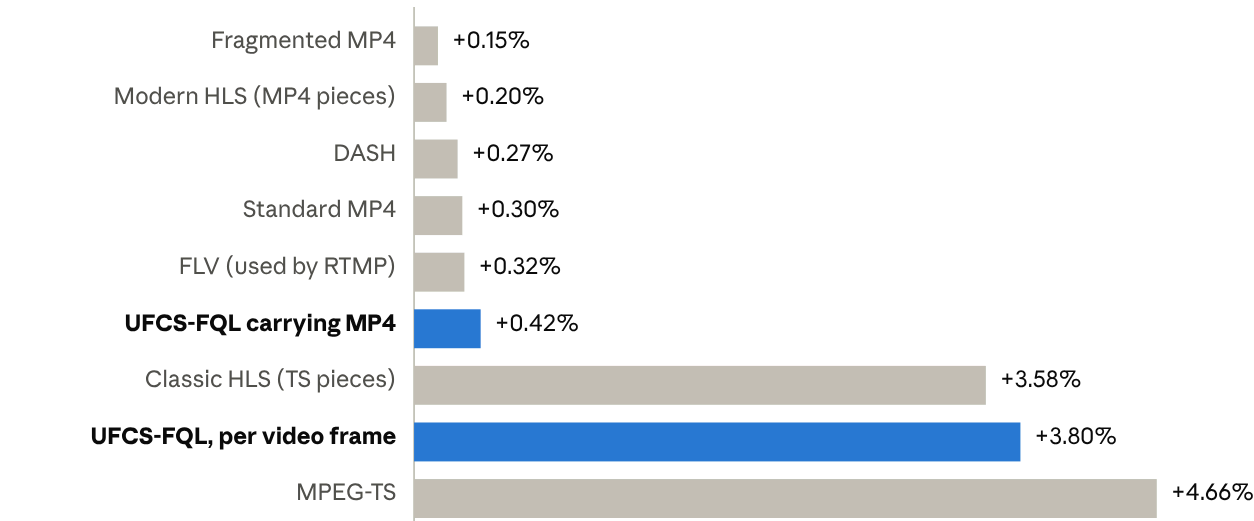
This is the protocol's main strength: urgent pieces get their own lane, so they never wait behind a big download. The newest web standard (HTTP/3) solves the same problem a different way, so this gap narrows against services that already use it.

## Efficiency: how much wrapping is added

Every format wraps the video in labels so the receiver knows what each piece is. Less wrapping means less wasted space and bandwidth. When our protocol carries a modern MP4, it is as light as the best industry formats.

Our wrapping adds 0.4%, as light as MP4; old MPEG-TS adds 4.7%

Extra bytes added on top of the bare video and sound (same 10-second encode), %



Blue = QueryBook UFCS-FQL. Network (TCP/IP) overhead of about 6% applies equally to every row.

Measured in the lab on 2026-10-05 · one H.264 2.5 Mbps + AAC encode, repackaged without re-encoding

When our protocol sends every single video frame as its own piece, for the lowest live delay, the wrapping rises to 3.8%. That is still lighter than classic HLS and MPEG-TS. Each piece carries a small text description (where it came from, how urgent it is); trimming that would bring the live figure under 1%.

The network itself (TCP/IP) adds about 6% to everything, ours and the industry’s alike. In our earlier tests, 94% of what crossed the wire was useful video, and the delivery receipts sent back cost under 0.3%.

## Storage: how much disk space it needs

Storage is essentially the same: one hour of 720p video at 2.5 Mbps needs about 1.25 GB whichever way it is stored. The only meaningful saving is avoiding the old MPEG-TS box, which wastes about 4.6%.

Figures below are measured from the same 10-second encode, scaled to one hour.

| How it is stored                     | GB per hour | TB per 1,000 hours | vs the leanest |
|--------------------------------------|-------------|--------------------|----------------|
| Fragmented MP4 (modern HLS and DASH) | 1.251       | 1.25               | leanest        |

| How it is stored                        | GB per hour | TB per 1,000 hours | vs the leanest |
|-----------------------------------------|-------------|--------------------|----------------|
| DASH package                            | 1.252       | 1.25               | +0.1%          |
| Standard MP4                            | 1.253       | 1.25               | +0.2%          |
| UFCS-FQL frames carrying fragmented MP4 | 1.254       | 1.25               | +0.3%          |
| Classic HLS (MPEG-TS pieces)            | 1.294       | 1.29               | +3.4%          |
| MPEG-TS single file                     | 1.307       | 1.31               | +4.5%          |

Normally you would store the plain MP4 and only wrap it in our frames while it travels, so our protocol adds nothing to the disk bill.

Two things cut storage much more than the delivery choice:

- **Newer shrinker.** AV1 typically needs about 30 to 50% less space than H.264 for the same picture (an industry rule of thumb, not measured here).
- **One copy instead of two.** Many services store an HLS copy and a DASH copy. Our receiving node rebuilds both from one stored copy, which can halve that part of the bill. The industry's CMAF format does the same.

## Accuracy: does what arrives match what was sent?

Picture accuracy is identical, because the same shrinkers do the work. Delivery accuracy is where our protocol does better: every piece is checked, and the whole file is proven identical on arrival.

**Picture accuracy** (how close the delivered picture is to the original master). These are the same numbers the industry would get with the same settings:

| Footage, at 2.5 Mbps                  | H.264 | H.265 | AV1   |
|---------------------------------------|-------|-------|-------|
| Studio graphics (SSIM, 1.0 = perfect) | 0.970 | 0.970 | 0.975 |
| Fast motion (SSIM)                    | 0.855 | 0.913 | 0.917 |

**Delivery accuracy** (did the bytes arrive untouched):

| Check                         | Industry web streaming                                         | QueryBook UFCS-FQL                                                  |
|-------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------|
| Each piece checked on arrival | Only by the network's basic 16-bit check, unless HTTPS is used | Yes, a 32-bit fingerprint on every piece                            |
| Whole file proven identical   | Not by default in HLS or DASH                                  | Yes, a SHA-256 fingerprint; 15 of 15 test transfers matched exactly |
| Proof of who sent it          | HTTPS proves the server, not the file                          | Optional digital signature on every piece                           |
| A damaged piece               | Can stall or break the player                                  | Dropped and skipped; the rest of the stream carries on              |
| Data captions and facts       | Passed through as-is                                           | Each fact re-checked against its fingerprint on arrival             |

In practice, a service using HTTPS everywhere already gets strong protection in transit. Our protocol's extra value is the per-file proof and signature, which matter for archives, legal evidence and rights-managed content.

## Live video: how far behind real time

Our protocol moved live video across a long-distance line in about 0.14 seconds. Standard web streaming usually runs several seconds behind, mainly because it sends video in 2 to 6 second chunks and the viewer's player waits for a few of them.

| Method                               | Typical delay behind live       | Source of figure                       |
|--------------------------------------|---------------------------------|----------------------------------------|
| Classic HLS or DASH                  | 6 to 30 s                       | Industry typical (approximate)         |
| Low-latency HLS or DASH              | 2 to 5 s                        | Industry typical (approximate)         |
| RTMP                                 | 2 to 5 s                        | Industry typical (approximate)         |
| WebRTC (video calls)                 | under 0.5 s                     | Industry typical (approximate)         |
| QueryBook UFCS-FQL, network leg only | 0.14 s typical, 0.18 s worst 5% | Measured, 20 Mbps line, 80 ms each way |

Be careful comparing these: our 0.14 s covers only the trip between two machines. It leaves out the camera, the encoder, a delivery network and the viewer's player, which the industry figures include. Our end-to-end delay is likely around 1 second, closer to WebRTC than to HLS, but we have not yet measured it camera to screen. The industry figures above are general ranges from memory, not measured here.

## Where the industry is still better

The industry wins on reach and scale. Our protocol is a working prototype; web streaming runs on every phone, TV and browser today.

- **Plays everywhere.** Every device plays HLS or DASH. Viewers cannot play our frames directly, so our receiving node converts back to HLS or DASH for them.
- **Delivery networks (CDNs).** CDNs cache web video near viewers for very little cost. They do not understand our frames, so our protocol fits best between your own sites, not to each viewer.
- **Raw download speed.** On a single big file, plain web delivery was about 15% faster in our test.
- **Video calls.** WebRTC is the proven tool for sub-second two-way video at scale.
- **Proven at scale.** HLS and DASH serve millions of viewers every day. Our numbers come from a lab, on simulated lines, with synthetic test video.

The practical takeaway: use the industry formats for the last step to the viewer. Use our protocol where it shines: between studios, data centres and edge sites, for live feeds, mixed urgent and bulk traffic, and anywhere you need proof that a file arrived untouched.

## How we tested

All measurements ran in the lab on one machine, with simulated network lines (a fixed delay and a speed cap in both directions).

- **Test clip:** 10 seconds of 720p test pattern at 30 frames per second, H.264 at 2.5 Mbps plus AAC sound at 128 kbps. The same encode was repackaged into each format with no re-encoding, so only the packaging differs.
- **Speed test:** the same 3.5 MB file over a 50 Mbps line with 20 ms delay each way. Web delivery used a standard Python web server; ours used the lab's sender and receiver. Each was run 3 times; the runs agreed within 0.02 s.



- **Live and quality figures:** from the lab's earlier video transit report (in the repository at [ufcs-transport-lab/docs/video-transit-report.html](https://ufcs-transport-lab/docs/video-transit-report.html) ).
- **Industry live delay ranges** are general figures from memory, not measured here.

Limits: the test video is synthetic, there was no packet loss, and both test nodes ran on the same machine. Real footage and real networks will shift the numbers, though not which side wins each layer.